皖西学院·正常模式 · 答案与解析

一、简答题（4×5=20分）

1.

答案:

（1）冯·诺依曼体系结构的主要特点：

① **存储程序**：程序（指令序列）和数据预先存入存储器，机器运行时自动按序取出并执行，无需人工干预。 ② **指令和数据以同等地位存储**：指令和数据都以二进制形式存放在存储器中，计算机在形式上不区分它们，均按地址访问。 ③ **五大功能部件**：计算机由运算器、控制器、存储器、输入设备和输出设备五大部件组成。 ④ **按地址顺序执行**：指令按其在存储器中的地址顺序存放和执行，遇到转移指令时改变执行顺序。 ⑤ **以运算器为中心**（经典冯·诺依曼模型）：早期结构中各部件的信息流经过运算器（现代计算机已改为以存储器为中心）。

（2）CPU 的两大组成部件及功能：

- **运算器（ALU 及相关寄存器）**：负责对数据进行算术运算和逻辑运算，核心是算术逻辑单元（ALU），还包含累加器、通用寄存器等。
- **控制器**：负责协调和控制计算机各部件按序工作，包括取指令、分析指令（译码）、产生控制信号等，是指挥中心。

解析：本题考察对冯·诺依曼体系结构核心思想的准确回忆，关键得分点为"存储程序"和"指令与数据同等地位"两个区别性特征。CPU 组成须明确回答"运算器"和"控制器"，不能遗漏或混为"五大部件"。常见错误：将"存储程序"表述为"程序存储"（不够准确，原术语为"存储程序" Stored Program）；混淆"冯·诺依曼五大部件"与"CPU 内部组成"。

2.

答案:

比较项目	SRAM（静态随机存储器）	DRAM（动态随机存储器）
存储原理	利用触发器（双稳态电路）存储信息，只要不断电信息不会丢失	利用电容上的电荷存储信息，电容会漏电导致信息逐渐丢失
是否需要刷新	不需要刷新	需要定期刷新（通常 2ms 内刷新一遍）
读写速度	速度快（存取时间短）	速度较慢（存取时间较长）
集成度与成本	集成度低、功耗较大、成本高	集成度高、功耗低、成本较低

比较项目	SRAM（静态随机存储器）	DRAM（动态随机存储器）
主要用途	用于 CPU 高速缓存（Cache）	用于计算机主存（内存条）

解析：本题从四个维度考察 SRAM 与 DRAM 的区别，属于高频对照型记忆题。核心记忆线索：SRAM = 触发器 = 不刷新 = 快 = 贵 = Cache；DRAM = 电容 = 要刷新 = 慢 = 便宜 = 主存。常见错误：混淆两者存储原理（将 SRAM 说成电容存储）；遗漏"刷新"这一 DRAM 的关键特征；搞反速度或用途。

3.

答案：

（1）中断响应过程（由硬件自动完成）：

① **关中断**：防止在保存断点过程中被新的中断打断（保证原子性）。 ② **保存断点**：将程序的返回地址（PC）及程序状态字（PSW）等压入堆栈或保存到指定寄存器。 ③ **引出中断服务程序**：通过中断向量表（或中断识别）找到中断服务程序的入口地址，装入 PC，从而转移到中断服务程序执行。

（2）中断处理流程（由中断服务程序完成）：

① **保护现场**：保存通用寄存器等内容，以便返回后恢复。 ② **中断服务（中断处理）**：执行具体的中断处理逻辑（如读取外设数据）。 ③ **恢复现场**：恢复被保护的寄存器内容。 ④ **开中断并中断返回**：恢复中断允许状态，执行中断返回指令，恢复断点（PC 和 PSW），返回被中断的程序继续执行。

（3）分界点： 中断响应是硬件自动完成的工作（关中断→保存断点→获取入口地址），中断处理是软件（中断服务程序）完成的工作（保护现场→中断服务→恢复现场→中断返回）。两者的分界点是**获取中断服务程序入口地址并转入执行的那一刻**——此前的操作由硬件完成，此后由软件执行。

解析：本题要求区分"硬件完成的响应过程"与"软件完成的处理流程"，关键得分点在于明确两者的分界。常见错误：将"保护现场"归入中断响应阶段（实际属中断处理）；遗漏"关中断"步骤；混淆中断响应的三个硬件步骤。

4.

答案：

（1）DMA 方式的工作原理：

DMA（直接存储器存取）方式允许外部设备与主存之间**直接进行数据交换**，数据传送过程**不需要 CPU 干预**，而是由专门的 DMA 控制器（DMAC）控制总线完成。

工作过程分三个阶段： - **预处理**：CPU 执行指令对 DMA 控制器进行初始化设置（如传送方向、数据块起始地址、数据块长度等），设置完后 CPU 继续执行原程序。 - **数据传送**：DMA 控制器接管

总线控制权，直接控制设备与主存之间的数据传送，CPU 不参与。 - **后处理**：数据传送结束后，DMA 控制器向 CPU 发中断信号，CPU 响应中断进行结束处理（如校验数据、释放资源等）。

（2）周期挪用（周期窃取）：

DMA 控制器在数据传送时需要占用总线周期。当 CPU 不需要访问主存时，DMA 直接使用总线；当 CPU 和 DMA 同时需要访问主存时，DMA 从 CPU 占用的存储周期中**"挪用"（窃取）一个或几个存储周期**用于数据传送，CPU 暂停一个周期。这种不停止 CPU 工作、仅占用个别总线周期的方式称为周期挪用（周期窃取）。

作用：周期挪用使 DMA 传送对 CPU 的影响最小化——CPU 无需长时间停止工作，只是在每次 DMA 传送时短暂让出一个总线周期，从而在保证数据高速传送的同时最大化 CPU 的利用率。

解析：本题考察 DMA 的核心概念及周期挪用机制。关键得分点：①数据传送不需 CPU 干预，由 DMAC 直接控制总线；②三个阶段（预处理、数据传送、后处理）；③周期挪用的含义是 DMA 在 CPU 不访问主存或争用时"窃取"个别总线周期。常见错误：将 DMA 说成"完全不占用 CPU"（后处理阶段仍需 CPU 中断参与）；混淆周期挪用与 CPU 停机访问（CPU 停止法是另一种 DMA 传送方式）。

二、计算题（4×10=40分）

1.

小题	结果
(1) [x]_变补	\texttt{00,1011}
(1) [y]_变补	\texttt{00,0111}
(2) x + y 变形补码和	\texttt{01,0010}，双符号位 `01` → **上溢（溢出）**
(3) [-y]_变补	\texttt{11,1001}
(3) x - y 变形补码差	\texttt{00,0100}，双符号位 `00` → **不溢出**

2.

项目	值
对阶右移位数	3（A 右移 3 位对齐到 E=129）
求和结果符号	正（0）
规格化方向	右规 1 位（E: 129→130）
舍入判定	guard=1, round=1, sticky=1 → 向上进位
二次右规	否

项目	值
最终 IEEE754 机器数	**0x41080000**

3.

项目	结果
命中率 h	0.95
缺失率 h_{miss}	0.05
AMAT	4 个时钟周期
Cache 效率 e	0.5000
$AMAT_2$（增设 L2 后）	2.8 个时钟周期
是否降低	是（4 → 2.8）

4.

扩展操作码编码——零地址指令最大条数计算 · 解答

问题一（3 分）— 扩展操作码编码基本原理

扩展操作码编码的基本原理：

在定长指令字中，操作码的长度不固定，而是根据指令地址码的个数动态扩展。操作码从固定位开始，向地址码字段（高位方向）逐级借用比特位来扩展。核心规则是：

- **固定操作码 + 可扩展前缀**：最低一级指令（三地址）使用最短的操作码，其余编码值用作"扩展前缀"，指示后续级别指令。
- **逐级扩展**：每减少一个地址码字段，操作码就向该字段扩展（借用该地址码的位数），从而获得更大的操作码空间。
- **低级指令用尽后高级指令复用其编码前缀**：某一级指令占用完自己的编码空间后，未使用的编码前缀被下一级指令借用。

操作码如何向地址码字段扩展：

以本题为例，指令字长 16 位，地址码每位 4 位：

级别	地址码个数	操作码位数	地址码占用
三地址	3	4 位	3 × 4 = 12 位
二地址	2	4 + 4 = 8 位	2 × 4 = 8 位

级别	地址码个数	操作码位数	地址码占用
一地址	1	8 + 4 = 12 位	1 × 4 = 4 位
零地址	0	12 + 4 = 16 位	0 位

三地址指令操作码 4 位（0000～1111），用完后三地址最后一个编码值作为扩展前缀，向第一个地址码字段扩展 4 位，形成二地址操作码；依此类推，每级向地址码字段扩展 4 位。

为什么三地址指令的操作码位数最少?

三地址指令需要 3 个地址码字段（共 3 × 4 = 12 位），留给操作码的位数最少（16 − 12 = 4 位）。地址码个数越多，占用字长越多，操作码可用位数越少。因此三地址指令的操作码位数最少，仅 4 位。

问题二（5 分）— 逐级计算零地址指令最大条数

基本参数

- 指令字长 L = 16 位
- 每个地址码位数 a = 4 位
- 三地址指令：操作码 4 位，占满编码空间（2^4 = 16 条）
- 二地址指令条数 N2 = 200
- 一地址指令条数 N1 = 500

逐级扩展过程

【三地址级】

- 三地址操作码位数 = L − 3a = 16 − 12 = **4 位**
- 三地址编码空间 = 2^4 = **16 个编码**
- 三地址指令占满全部 16 个编码 → 进入二地址扩展

【二地址级】

- 二地址操作码空间 = 2^(L−2a) = 2^8 = **256 个编码槽**
- 二地址指令占用 N2 = **200 条**
- **剩余前缀1** = 256 − 200 = **56**
- （验证：56 > 0 √，且 56 不为 2 的幂 √）

【一地址级】

- 一地址指令空间 = 剩余前缀1 × 2^a = 56 × 16 = **896 个编码槽**
- 一地址指令占用 N1 = **500 条**
- **剩余前缀2** = 896 − 500 = **396**

· （验证：396 > 0 √，且 396 不为 2 的幂 √）

【零地址级】

· 零地址最大条数 N0_max = 剩余前缀2 × 2^a = 396 × 16 = **6336**

最终结果

$\boxed{N0_{max} = 6336 条}$

完整公式：

$N0_{max} = \left[\left(2^{L-2a} - N2\right) \times 2^{a} - N1\right] \times 2^{a} = \left[(256 - 200) \times 16 - 500\right] \times 16 = (896 - 500) \times 16 = 396 \times 16 = 6336$

场景规则验证

约束	检查	结果
N2 < 2^(L−2a)	200 < 256	√
剩余前缀1 > 0	56 > 0	√
剩余前缀2 > 0	396 > 0	√
N0_max = 剩余前缀2 × 2^a	6336 = 396 × 16	√
剩余前缀1 非 2 的幂	56	√
剩余前缀2 非 2 的幂	396	√

问题三（2 分）— N2 增至 210 的影响

将二地址指令条数从 N2 = 200 增加到 N2 = 210（其余条件不变）：

重新计算各级数值

量	N2 = 200（原）	N2 = 210（新）	变化
剩余前缀1	56	46	减少 10
剩余前缀2	396	236	减少 160
N0_max	6336	3776	**减少 2560**

变化方向与原因

变化方向：N0_max 减少。

原因（级联效应）：

1. N2 增加 10 → 剩余前缀1 减少 10（56 → 46）

2. 剩余前缀1 减少 10 → 一地址空间减少 10 × 2^a = 10 × 16 = 160 → 剩余前缀2 减少 160（396 → 236）
3. 剩余前缀2 减少 160 → 零地址最大条数减少 160 × 2^a = 160 × 16 = 2560（6336 → 3776）

结论： 在扩展操作码逐级扩展链中，上级（二地址）指令条数增加会挤压剩余前缀，该效应通过"×2^a"因子逐级放大，最终导致零地址指令最大条数减少。每多占用 1 个二地址编码槽，零地址指令将减少 $2^{(2a)} = 2^8 = 256$ 条。10 个二地址编码槽的增量导致零地址减少 10 × 256 = **2560 条**。

$\boxed{N0_{\max}$ 从 6336 减少到 3776 （减少 2560 条）$}$

三、分析题（2×10=20分）

1.

指令流水线性能分析 — 求解

> 以下数值均由 gen.py 计算产出，逐字对照 exec 数据流输出。

基本参数

参数	值
段数 k	5
各段时间	IF=10, ID=10, EX=20, MEM=10, WB=10（单位 ns）
瓶颈段	EX = 20 ns
指令条数 N	200

Q1（3分）

（1）确定流水线周期 T

流水线周期由**瓶颈段**（各段时间最大值）决定：

$T = \max(\Delta t_i) = \max(10, 10, 20, 10, 10) = 20$ ns

依据：各段在同一时钟周期内并行工作，流水线时钟必须迁就最慢的段，否则该段无法在单个周期内完成操作。因此 T = 瓶颈段时间 = 20 ns。

（2）总执行时间

总时间 = (k + N - 1) × T = (5 + 200 - 1) × 20 = 204 × 20 = 4080 ns

· 第 1 条指令需 5 个周期（k=5）才能走完全部段（**填充**阶段）。

· 之后每条指令只需 1 个周期（每周期流出 1 条）。

· 排空时间 = (k−1)×T = 4×20 = 80 ns，占比 = 80/4080 ≈ 1.96% < 5%。

> **总执行时间 = 4080 ns**

Q2（3分）

（1）实际吞吐率 TP

TP = \frac{N}{总时间} = 200/4080 = 0.04902 条/ns = 49.02 MIPS

（2）最大吞吐率 TP_max

$TP_{\max} = 1/T = 1/20 = 0.05$ 条/ns = 50.00 MIPS

（3）差距原因

实际吞吐率（49.02 MIPS）< 最大吞吐率（50.00 MIPS），TP/TP_max ≈ 0.9804。

原因：最大吞吐率假设流水线稳定运行（每周期恒流出 1 条指令）。但实际执行中，第一条指令需要 k=5 个周期才能通过全部段（**填充/排空阶段**），这 (k−1)=4 个周期没有指令流出，导致实际吞吐率低于理论最大值。当 N→∞ 时，排空占比趋近于 0，TP→TP_max。

Q3（2分）

加速比 S

顺序执行时，每条指令需经过全部 5 段，单条时间 = ΣΔt = 10+10+20+10+10 = 60 ns。

顺序执行总时间 = $N \times \sum \Delta t_i = 200 \times 60 = 12000$ ns

S = \frac{顺序时间}{流水线时间} = 12000/4080 = 2.941

> **加速比 S ≈ 2.94**

验证：1 < S=2.94 < k=5（流水线优于顺序执行，但未达理想 k 倍加速，因各段时间不等、存在排空开销）。

Q4（2分）

（1）优化后指标

将瓶颈段 S_3（EX, 20 ns）拆分为 S_{3a} 和 $S_{3\beta}$ 各 10 ns，流水线变为 6 段、各段均为 10 ns。

指标	优化前	优化后
段数	k=5	k'=6
流水线周期	T=20 ns	**T'=10 ns**
总执行时间	4080 ns	**(6+200−1)×10 = 2050 ns**
吞吐率	49.02 MIPS	**200/2050 = 97.56 MIPS**

计算： - T' = max(各新段) = 10 ns - 总时间' = (k'+N−1)×T' = (6+200−1)×10 = 2050 ns - TP' = N/总时间' = 200/2050 = 0.09756 条/ns = **97.56 MIPS**

（2）优化前后对比及段数对排空的影响

吞吐率变化：TP 从 49.02 → 97.56 MIPS，提升约 99%（接近翻倍）。原因是周期减半（T: 20→10 ns）使最大吞吐率翻倍（1/T': 50→100 MIPS），而段数仅增加 1 段，排空时间的微量增加远不抵周期减半带来的收益。

段数对排空时间的影响:

	优化前	优化后
排空时间 = (k−1)×T	4×20 = 80 ns	5×10 = 50 ns
排空占比	80/4080 ≈ 1.96%	50/2050 ≈ 2.44%

段数增加（5→6）会使排空周期数从 4 增到 5（多 1 个周期），但由于周期 T' 同时减半（20→10 ns），排空的**绝对时间反而减少**（80→50 ns）。排空占比虽从 1.96% 略升至 2.44%，但影响极小。

> **结论**：段数增加会增多排空周期数（+1），但不会降低整体性能——瓶颈段消除带来的周期减半使总执行时间大幅缩短（4080→2050 ns），吞吐率近乎翻倍。排空时间的绝对值反而下降。

2.

子问	答案
问题1	三种方式原理与优缺点见上（链式查询：简单但固定优先级、断链敏感；计数器定时：可调优先级但线数多；独立请求：响应快、灵活但线数多、仲裁器复杂）
问题2	**总线带宽 = 2132 MB/s**（= 4 B × 533 MHz）
问题3	**选择独立请求方式**（响应快 + 优先级可动态调整，满足高响应速度和高灵活性需求）

四、设计题（2×10=20分）

1.

问题	答案
Q1 各子字段位数	第一组 3 位、第二组 2 位、第三组 2 位、第四组 2 位
Q1 操作控制字段总位数	9 位
Q2 P 字段位数	2 位
Q2 微指令字长	20 位（操作控制 9 + P 字段 2 + 下地址 9）
Q3 控存最大容量	**10240 bit = 10 Kbit**

2.

小题	结论
(1)	字位同时扩展；位扩展数 = 2，字扩展数 = 4，芯片总数 = **8 片**
(2)	片内地址 = 14 位（A13～A0）；片选地址 = 2 位（A15～A14）；DRAM 对外地址引脚 = **7 根**
(3)	2-4 译码，A15A14 分 4 组：00/01/10/11，地址范围 0x0~0x3FFF / 0x4000~0x7FFF / 0x8000~0xBFFF / 0xC000~0xFFFF
(4)	刷新行数 = **128 行**；异步刷新周期 ≈ **15.625 μs**